



NANYANG
TECHNOLOGICAL
UNIVERSITY
SINGAPORE

*Addressing Issues in Generative AI System
Design and Deployment*

Part 2-Chapter 3: Secure Development and Testing

Ong Chin Ann

Lecturer

College of Computing & Data Science
(chinann.ong@ntu.edu.sg)



COPYRIGHT STATEMENT

- All course materials, including but not limited to, lecture slides, handout and recordings, are for your own educational purposes only. **All the contents of the materials are protected by copyright, trademark or other forms of proprietary rights.**
- All rights, title and interest in the materials are owned by, licensed to or controlled by the University, unless otherwise expressly stated. **The materials shall not be uploaded, reproduced, distributed, republished or transmitted in any form or by any means, in whole or in part, without written approval from the University.**
- You are also not allowed to take any photograph, film, audio record or other means of capturing images or voice of any contents during lecture(s) and/or tutorial(s) and reproduce, distribute and/or transmit any form or by any means, in whole or in part, without the written permission from the University.
- Appropriate action(s) will be taken against you including but not limited to disciplinary proceeding and/or legal action if you are found to have committed any of the above or infringed the University's copyright.

Part 2-Topic 3: Secure Development and Testing

- What are the common practices for secure development?
- What are the recommended guideline/framework for secure development?
- How do we recognize potential risks and vulnerabilities poses in our AI powered system?
- What are the mitigation strategies to address these potential risks and vulnerabilities?
- How do we conduct systematic and structured testing against the developed AI powered system?

Secure Development

NCSC Guidelines for Secure AI System Development

- Secure your supply chain
- Identify, track, and protect your assets
- Document your data, models, and prompts
- Manage your technical debt

Secured Supply Chain

- assess and monitor the security of your AI supply chains across a system's life cycle
- Require suppliers to adhere to the same standards your own organisation applies to other software.
 - Otherwise, follow your existing risk management policies
- Verified commercial, open source, and other third-party developers to ensure robust security in your systems
- Be prepared to failover to alternate solutions for mission-critical systems, if security criteria are not met.
- Example: NCSC 12 Principles of [Supply Chain Guidance](#)



Secured Supply Chain

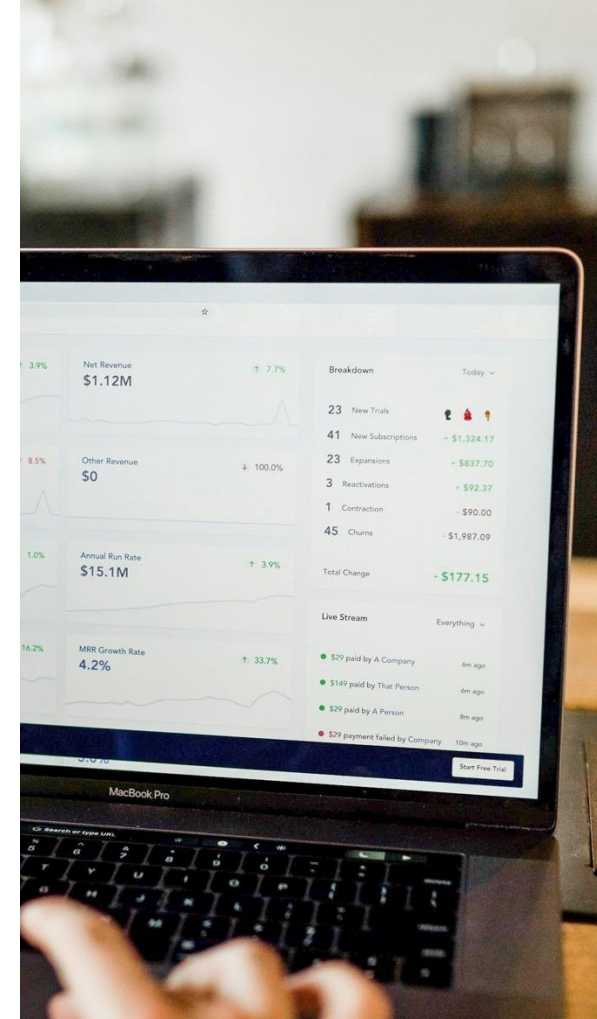
NCSC 12 Principles of [Supply Chain Guidance](#)

- Understand Risks
 1. Understand what needs to be protected and why
 2. Know who your suppliers are and build an understanding of what their security looks like
 3. Understand the security risk posed by your supply chain
- Establish control
 4. Communicate your view of security needs to your suppliers
 5. Set and communicate minimum security requirements for your suppliers
 6. Build security considerations into your contracting processes and require that your suppliers do the same
 7. Meet your own security responsibilities as a supplier and consumer
 8. Raise awareness of security within your supply chain
 9. Provide support for security incidents
- Check your arrangements
 10. Build assurance activities into your supply chain management
- Continuous improvement
 11. Encourage the continuous improvement of security within your supply chain
 12. Build trust with suppliers



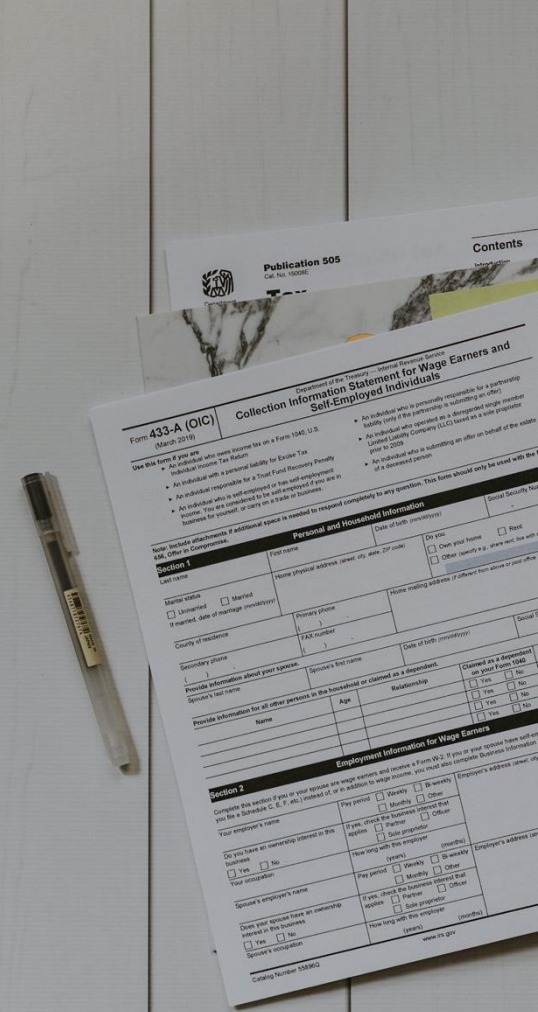
Identify, Track, and Protect Assets

- Understand the value of your organization AI-related assets (models, data, prompt, software, documentation, logs, and assessments)
- Recognize its vulnerabilities and threats.
- Implement access control to protect its confidentiality, integrity, and availability (CIA)
- Prepare the processes and tools to track, authenticate, version control, and perform data recovery in the event of compromise.



Documentation

- Document the creation, operation, and life cycle management of any models, datasets and meta or system-prompts.
 - Sources of training data (including fine-tuning data and human or other operational feedback),
 - intended scope and limitations,
 - guardrails,
 - cryptographic hashes or signatures,
 - retention time,
 - suggested review frequency and potential failure modes.
- Crucial and helps to ensure transparency and accountability



Manage Technical Debt

- **Technical Debt** is where engineering decisions that fall short of best practices to achieve short-term results are made, at the expense of longer-term benefits.
- should be managed from the earliest stages of development.
- Managing technical debt in AI system is far more challenging than typical or conventional software system.
 - Due to rapid development cycle
 - Lack of well-established protocols and interfaces.

Every technology provider must take ownership at the executive level to ensure their products are secure by design

WHAT ARE THE RECOMMENDED GUIDELINE FOR SECURE DEVELOPMENT?

NIST 800-218 Secure Software Development Framework (SSDF) V 1.1

- Published by the National Institute of Standards and Technology (NIST), US. (Feb 2022)
- Recommendations for Mitigating the Risk of **Software Vulnerabilities**
- A core set of high-level secure software development practices that can be integrated into each SDLC implementation
- Aims to :
 - Prepare the Organization (PO)
 - Protect the Software (PS)
 - **Produce Well-Secured Software (PW)**
 - Respond to Vulnerabilities (RV)
- <https://csrc.nist.gov/projects/ssdf>

NIST Special Publication 800-218

Secure Software Development Framework (SSDF) Version 1.1:

*Recommendations for Mitigating
the Risk of Software Vulnerabilities*

Murugiah Souppaya
Karen Scarfone
Donna Dodson

This publication is available free of charge from:
<https://doi.org/10.6028/NIST.SP.800-218>

NIST
National Institute of
Standards and Technology
U.S. Department of Commerce

NIST 800-218 Secure Software Development Framework (SSDF) V 1.1

- The SSDF does not prescribe specifically how to implement each practice.
- The focus is on the outcomes of the practices rather than on the tools, techniques, and mechanisms to do so.
 - Technical details on how to mitigate or remediate specific vulnerability can refer low-level guideline / framework such as MITRE ATT&CK / MITRE ATLAS / CVEs
- This means that the SSDF can be used by organizations in any sector or community, regardless of size or cybersecurity sophistication. It can also be used for any type of software development, regardless of technology, platform, programming language, or operating environment.
- <https://csrc.nist.gov/projects/ssdf>

NIST Special Publication 800-218

Secure Software Development Framework (SSDF) Version 1.1:

*Recommendations for Mitigating
the Risk of Software Vulnerabilities*

Murugiah Souppaya
Karen Scarfone
Donna Dodson

This publication is available free of charge from:
<https://doi.org/10.6028/NIST.SP.800-218>

NIST
National Institute of
Standards and Technology
U.S. Department of Commerce

NIST 800-218 Secure Software Development Framework (SSDF) V 1.1

NIST SP 800-218

SSDF Version 1.1

- 4 Key objectives / aims
 - PO, PS, PW, RV
- 18 practices
 - Sub divided into tasks with notional implementation examples and references

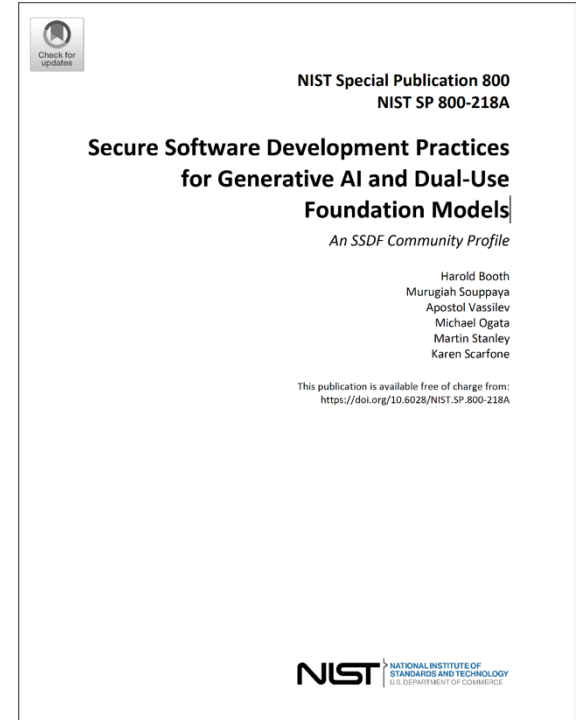
Table 1: The Secure Software Development Framework (SSDF) Version 1.1

Practices	Tasks	Notional Implementation Examples	References
Prepare the Organization (PO)			
Define Security Requirements for Software Development (PO.1): Ensure that security requirements for software development are known at all times so that they can be taken into account throughout the SDLC. The amount of security information that can be collected once and shared. This includes requirements from internal sources (e.g., the organization's policies, business objectives, and risk management strategy) and external sources (e.g., applicable laws and regulations).	PO.1.1: Identify and document all security requirements for the organization's software development infrastructures and processes, and maintain the requirements over time.	Example 1: Define policies for securing software development infrastructures and their components, including development endpoints, throughout the SDLC and maintaining that security. Example 2: Define policies for securing software development processes throughout the SDLC and maintaining that security, including for open-source and other third-party software components utilized by software being developed. Example 3: Review and update security requirements at least annually, or sooner if there are new requirements from internal or external sources, or a major security incident targeting software development infrastructure has occurred. Example 4: Educate affected individuals on impending changes to requirements.	BSA&SS: SM.3, DE.1, IA.1, IA.2 ES&MM: CP.1.1, CP.1.3, SR.1, SR.2, SE.1.2, SE.2.6 E014028: 4e(i) EC&A&S: SM-7, SM-9 NISTCSF: ID.OV-3 CWAS&SPASVS: 1.1.1 CWAS&SPASVS: 1.1.10 CWAS&SPASVS: PC.1-A, PC.1-B, PC.2-A PCIS&LC: 2.1, 2.2 SCF&SSD: Planning the Implementation and Deployment of Secure Development Practices SP800052: SA-1, SA-8, SA-15, SR-3 SP800109: 3.1.2, 3.2.1, 3.2.2, 3.3.1, 3.4.2, 3.4.3 SP800101: SA-1, SA-8, SA-15, SR-3 SP800101: TD.14, KD003, KD039, KD044, KD167, KD168, KD177, KD211, KD260, KD261, KD262, KD264, SD010, SD037, SD066, AD033, AD123, AD151
	PO.1.2: Identify and document all security requirements for organization-developed software to meet, and maintain the requirements over time.	Example 1: Define policies that specify risk-based software architecture and design requirements, such as making code modular to facilitate code reuse and updates; isolating security components from other components during execution; avoiding undocumented commands and settings; and providing features that will all software acquires with the secure deployment, operation, and maintenance of the software. Example 2: Define policies that specify the security requirements for the organization's software, and verify compliance at key points in the SDLC (e.g., classes of software flaws verified by gates, responses to vulnerabilities discovered in released software). Example 3: Analyze the risk of applicable technology stacks (e.g., languages, environments, deployment models), and recommend or require the use of stacks that will reduce risk compared to others. Example 4: Define policies that specify what needs to be archived for each software release (e.g., code, package files, third-party libraries, documentation, data inventory) and how long it needs to be retained based on the SDLC model, software end-of-life, and other factors. Example 5: Ensure that policies cover the entire software life cycle, including notifying users of the impending end of software support and the date of software end-of-life. Example 6: Review all security requirements at least annually, or sooner if there are new requirements from internal or external sources, a major vulnerability is discovered in released software, or a major security incident targeting organization-developed software has occurred. Example 7: Establish and follow processes for handling requirement exception requests, including periodic reviews of all approved exceptions.	BSA&SS: SC.1-1, SC.2, PD.1-1, PD.1-2, PD.1-3, PD.2-2, SI, PA, CS, AA, LO, EE ES&MM: SM.1.1, SM.1.4, SM.2, CP.1.1, CP.1.2, CP.1.3, CP.2.1, CP.2.3, AM.1.2, SFD.1.1, SFD.2.1, SFD.3.2, SR.1.1, SR.1.3, SR.2.2, SR.3.3, SR.4 E014028: 4e(i) EC&A&S: BR-3, SR-4, SR-4, SR-4, SR-4 IS027034: 7.3.2 MS&SL: 2.5 NISTCSF: ID.OV-3 CWAS&SPASVS: 1.1.2 CWAS&SPASVS: PC.1-A, PC.1-B, PC.2-A, PC.3-A, SR.1-A, SR.1-B, SR.2-B, SA.1-B, RI.1-A PCIS&LC: 2.1, 2.2, 2.3, 3.3 SCF&SSD: Establish Coding Standards and Conventions SP800053: SA-4, SA-6(3), SA-15, SR-3 SP800109: 3.1.2, 3.2.1, 3.3.1 SP800101: SA-4, SA-15, SR-3 SP800101: TD.14, KD003, KD039, KD044, KD167, KD168, KD177, KD211, KD260, KD261, KD262, KD264, SD010, SD037, SD066, AD033, AD123, AD151
	PO.1.3: Communicate requirements to all third parties who will provide commercial software components to the organization for reuse by the organization's own software. (Formerly PW.3.1)	Example 1: Define a core set of security requirements for software components, and include it in acquisition documents, software contracts, and other agreements with third parties. Example 2: Define security-related criteria for selecting software; the criteria can include the third party's vulnerability disclosure program and product security incident response capabilities or the third party's adherence to organization-defined practices. Example 3: Require third parties to attest that their software complies with the organization's security requirements.	BSA&SS: SM.1, SM.2, SM.2-1, SM.2-4 ES&MM: CP.2-A, CP.2.2, SR.2.2, SR.2.3 E014028: 4e(i), 4e(ii) EC&A&S: 19.21 EC&A&S: SM-6, SM-10 MS&SL: 7 NISTCSF: ID.SC-3 CWAS&SPASVS: SR.3-A



NIST 800-218A Secure Software Development Practices for Generative AI and Dual-Use Foundation Models

- Augments the secure software development practices and tasks defined in Secure Software Development Framework (SSDF) version 1.1
- Adding practices, tasks, recommendations, considerations, notes, and informative references that are specific to AI model development throughout the software development life cycle.
- Intended to be useful to
 - the producers of AI models,
 - the producers of AI systems that use those models, and
 - the acquirers of those AI systems.
- <https://csrc.nist.gov/pubs/sp/800/218/a/final>



NIST 800-218A Secure Software Development Practices for Generative AI and Dual-Use Foundation Models

- Similar to SSDF V1.1
- 4 Key objectives / aims
 - PO, PS, PW, RV
- 18 practices
 - Sub divided into tasks with notional implementation examples and references
- Added priorities (High, Medium, Low)
- Added Recommendation (“R”), Consideration (“C”) and Notes Specific to AI Model Development (“N”) tags.

NIST SP 800-218A
July 2024

Secure Software Development Practices for
Generative AI and Dual-Use Foundation Models

Note: This Profile supplements what SSDF version 1.1 [6] already includes and is intended to be used in conjunction with it, not on its own. As a reminder, the deployment and operation of AI systems with AI models are out of the Profile’s scope, as are most parts of the data governance and management life cycle.

There are gaps in the numbering of some SSDF practices and tasks. For example, the PW.4 practice has three tasks: PW.4.1, PW.4.2, and PW.4.4. PW.4.3 was a task in SSDF version 1.0 that was moved elsewhere for version 1.1, so its ID was not reused.

Table 1. SSDF Community Profile for AI Model Development

Practice	Task	Priority	Recommendations [R], Considerations [C], and Notes [N] Specific to AI Model Development	Informative References
Prepare the Organization (PO)				
Define Security Requirements for Software Development (PO.1): Ensure that security requirements for software development are known at all times so that they can be taken into account throughout the software development life cycle (SDLC) and duplication of effort can be minimized because the requirements information can be collected once and shared. This includes requirements from internal sources (e.g., the organization’s policies, business objectives, and risk management strategy) and external sources (e.g., applicable laws and regulations).	PO.1.1: Identify and document all security requirements for the organization’s software development infrastructures and processes, and maintain the requirements over time.	High	R1: Include AI model development in the security requirements for software development infrastructure and processes. R2: Identify and select appropriate AI model architectures and training techniques in accordance with recommended practices for cybersecurity, privacy, and reproducibility.	AI RMF: Map 1.3, 1.5, 1.6
	PO.1.2: Identify and document all security requirements for organization-developed software to meet, and maintain the requirements over time.	High	R1: Organizational policies should support all current requirements specific to AI model development security for organization-developed software. These requirements should include the areas of AI model development, AI model operations, and data science. Requirements may come from many sources, including laws, regulations, contracts, and standards. C1: Consider reusing or expanding the organization’s existing data classification policy and processes. N1: Possible forms of AI model documentation include data, model, and system cards.	AI RMF: Govern 1.1, 1.2, 3.2, 4.1, 5.1, 6.1; Map 1.1
	PO.1.3: Communicate requirements to all third parties who will provide commercial software components to the organization for use by the organization’s own software. [Modified from SSDF 1.1]	Medium	R1: Include AI model development security in the requirements being communicated for third-party software components.	AI RMF: Map 4.1, 4.2 OWASP: LLM05-1

NIST 800-218A Secure Software Development Practices for Generative AI and Dual-Use Foundation Models

Prepare the Organization (PO)

- Define Security Requirements for Software Development (PO.1)
- Implement Roles and Responsibilities (PO.2)
- Implement Supporting Toolchains (PO.3)
- Define and Use Criteria for Software Security Checks (PO.4)
- Implement and Maintain Secure Environments for Software Development (PO.5)

Protect Software (PS)

- Protect All Forms of Code and Data from Unauthorized Access and Tampering (PS.1)
- Provide a Mechanism for Verifying Software Release Integrity (PS.2)
- Archive and Protect Each Software Release (PS.3)

NIST 800-218A Secure Software Development Practices for Generative AI and Dual-Use Foundation Models

Produce Well-Secured Software (PW)

- Design Software to Meet Security Requirements and Mitigate Security Risks (PW.1)
- Review the Software Design to Verify Compliance with Security Requirements and Risk Information (PW.2)
- Confirm the Integrity of Training, Testing, Fine-Tuning, and Aligning Data Before Use (PW.3)
- Reuse Existing, Well-Secured Software When Feasible Instead of Duplicating Functionality (PW.4)
- Create Source Code by Adhering to Secure Coding Practices (PW.5)
- Configure the Compilation, Interpreter, and Build Processes to Improve Executable Security (PW.6)
- Review and/or Analyze Human-Readable Code to Identify Vulnerabilities and Verify Compliance with Security Requirements (PW.7)
- Test Executable Code to Identify Vulnerabilities and Verify Compliance with Security Requirements (PW.8)
- Configure Software to Have Secure Settings by Default (PW.9)

NIST 800-218A Secure Software Development Practices for Generative AI and Dual-Use Foundation Models

Respond to Vulnerabilities (RV)

- Identify and Confirm Vulnerabilities on an Ongoing Basis (RV.1)
- Assess, Prioritize, and Remediate Vulnerabilities (RV.2)
- Analyze Vulnerabilities to Identify Their Root Causes (RV.3)

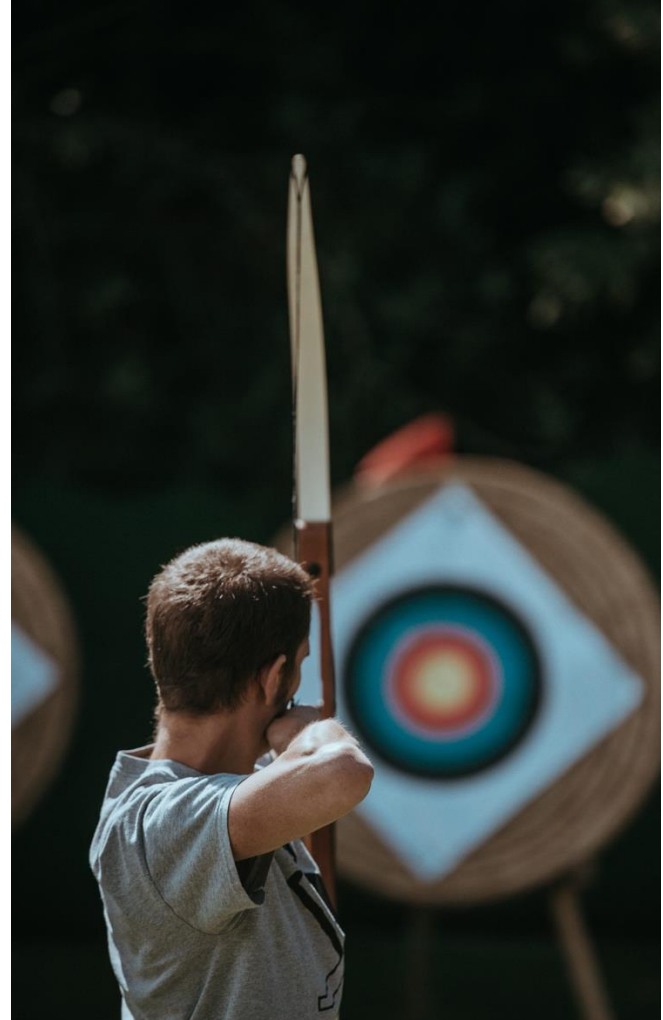
SSRF guidelines are quite generics...

**HOW DO WE RECOGNIZE POTENTIAL RISK AND
VULNERABILITY POSES IN OUR PRODUCTS?**



Recognizing Risks

- One way to recognize and classify risk and vulnerability is through the tactics of adversarial attack.
- Think like an attacker, what action(s) would you take if you wish to exploit a system?
- Maybe you will first try to understand the architecture of the system, collect some information about the system, looking for loop holes and targets etc...



ATLAS Matrix

The ATLAS Matrix below shows the progression of tactics used in attacks as columns from left to right, with ML techniques belonging to each tactic below. & indicates an adaption from ATT&CK. Click on the blue links to learn more about each item, or search and view ATLAS tactics and techniques using the links at the top navigation bar. View the ATLAS matrix highlighted alongside ATT&CK Enterprise techniques on the [ATLAS Navigator](#).

Reconnaissance&	Resource Development&	Initial Access&	AI Model Access	Execution&	Persistence&	Privilege Escalation&	Defense Evasion&	Credential Access&	Discovery&	Collection&	AI Attack Staging	Command and Control&	Exfiltration&	Impact&
6 techniques	12 techniques	6 techniques	4 techniques	4 techniques	4 techniques	2 techniques	8 techniques	1 technique	7 techniques	3 techniques	4 techniques	1 technique	5 techniques	7 techniques
Search Open Technical Databases& Search Open AI Vulnerability Analysis Search Victim-Owned Websites& Search Application Repositories Active Scanning& Gather RAG-Indexed Targets	Acquire Public AI Artifacts Obtain Capabilities& Develop Capabilities& Acquire Infrastructure Publish Poisoned Datasets Poison Training Data Establish Accounts& Publish Poisoned Models Publish Hallucinated Entities LLM Prompt Crafting Retrieval Content Crafting Stage Capabilities&	AI Supply Chain Compromise Valid Accounts& Evade AI Model Exploit Public-Facing Application& Phishing& Drive-by Compromise&	AI Model Inference API Access AI-Enabled Product or Service Physical Environment Access Full AI Model Access	User Execution& Command and Scripting Interpreter& LLM Prompt Injection LLM Plugin Compromise	Poison Training Data Manipulate AI Model LLM Prompt Self-Replication RAG Poisoning	LLM Plugin Compromise LLM Jailbreak	Evade AI Model LLM Jailbreak LLM Trusted Output Components Manipulation LLM Prompt Obfuscation False RAG Entry Injection Impersonation& Masquerading& Corrupt AI Model	Unsecured Credentials&	Discover AI Model Ontology Discover AI Model Family Discover AI Artifacts Discover LLM Hallucinations Discover AI Model Outputs Discover LLM System Information Cloud Service Discovery&	AI Artifact Collection Data from Information Repositories& Data from Local System&	Create Proxy AI Model Manipulate AI Model Verify Attack Craft Adversarial Data	Reverse Shell	Exfiltration via AI Inference API Exfiltration via Cyber Means Extract LLM System Prompt LLM Data Leakage LLM Response Rendering	Evade AI Model Denial of AI Service Spamming AI System with Chaff Data Erode AI Model Integrity Cost Harvesting External Harms Erode Dataset Integrity

<https://atlas.mitre.org/matrices/ATLAS>

ATLAS MITRE

ATLAS Matrix

The ATLAS Matrix below shows the progression of tactics used in attacks as columns from left to right, with ML techniques belonging to each tactic below. & indicates an adaption from ATT&CK. Click on the blue links to learn more about each item, or search and view ATLAS tactics and techniques using the links at the top navigation bar. View the ATLAS matrix highlighted alongside ATT&CK Enterprise techniques on the ATLAS Navigator.

Reconnaissance ^{&} 6 techniques	Resource Development ^{&} 12 techniques	Initial Access ^{&} 6 techniques	AI Model Access ^{&} 4 techniques	Execution ^{&} 4 techniques	Persistence ^{&} 4 techniques	Privilege Escalation ^{&} 2 techniques	Defense Evasion ^{&} 3 techniques	Credential Access ^{&} 1 technique	Discovery ^{&} 7 techniques	Collection ^{&} 3 techniques	AI Attack Staging ^{&} 4 techniques	Command and Control ^{&} 1 technique	Exfiltration ^{&} 5 techniques	Impact ^{&} 7 techniques
Search Open Technical Databases ^{&}	Acquire Public AI Interfaces ^{&}	AI Supply Chain Compromise	AI Model Inference API Access	User Execution ^{&}	Poison Training Data	LLM Plugin Compromise	Evaide AI Model	Unsecured Credentials ^{&}	Discover AI Model Ontology	AI Artifact Collection	Create Proxy AI Model	Reverse Shell	Exfiltration via AI Inference API	Evaide AI Model
Search Open AI Vulnerability Analyses	Obtain Capabilities ^{&}	Vendor Accounts ^{&}	AI Enabled Product or Service	Command and Control Interceptor ^{&}	Manipulate AI Model	LLM Jailbreak	LLM Jailbreak		Discover AI Model Family	Manipulate AI Model	Manipulate AI Model		Exfiltration via AI Inference API	Control of AI Service
Search Vendor-Owned Websites ^{&}	Develop Capabilities ^{&}	Evaide AI Model	Physical Environment Access	LLM Prompt Injection	LLM Prompt Injection	LLM Prompt Injection	LLM Prompt Output Components Manipulation		Discover AI Model	Data from Intermediate Repositories ^{&}	Verify Attack		Exfiltration via AI Inference API	Spanning AI System with Chat Data
Search Application Repositories	Acquire Infrastructure	Exploit Public-Facing Application ^{&}	Full AI Model Access	LLM Prompt Injection	LLM Prompt Injection	LLM Prompt Injection	LLM Prompt Output Components Manipulation		Discover LLM Hallucinations	Data from Local System ^{&}	Craft Adversarial Data		Exfiltration via AI Inference API	Spanning AI System with Chat Data
Active Scanning ^{&}	Public Poisoned Datasets	Phishing ^{&}		LLM Prompt Injection	LLM Prompt Injection	LLM Prompt Injection	LLM Prompt Output Components Manipulation		Discover AI Model				Exfiltration via AI Inference API	Spanning AI System with Chat Data
Establish Scanning Targets	Drive-by Compromise ^{&}	Drive-by Compromise ^{&}		LLM Prompt Injection	LLM Prompt Injection	LLM Prompt Injection	LLM Prompt Output Components Manipulation		Discover LLM Hallucinations				Exfiltration via AI Inference API	Spanning AI System with Chat Data
	Establish Accounts ^{&}			LLM Prompt Injection	LLM Prompt Injection	LLM Prompt Injection	LLM Prompt Output Components Manipulation		Discover AI Model				Exfiltration via AI Inference API	Spanning AI System with Chat Data
	Public Poisoned Models			LLM Prompt Injection	LLM Prompt Injection	LLM Prompt Injection	LLM Prompt Output Components Manipulation		Discover LLM Hallucinations				Exfiltration via AI Inference API	Spanning AI System with Chat Data
	Public Hallucinated Entities			LLM Prompt Injection	LLM Prompt Injection	LLM Prompt Injection	LLM Prompt Output Components Manipulation		Discover AI Model				Exfiltration via AI Inference API	Spanning AI System with Chat Data
	LLM Prompt Crafting			LLM Prompt Injection	LLM Prompt Injection	LLM Prompt Injection	LLM Prompt Output Components Manipulation		Discover LLM Hallucinations				Exfiltration via AI Inference API	Spanning AI System with Chat Data
	Retrieval Content Crafting			LLM Prompt Injection	LLM Prompt Injection	LLM Prompt Injection	LLM Prompt Output Components Manipulation		Discover AI Model				Exfiltration via AI Inference API	Spanning AI System with Chat Data
	Stage Capabilities ^{&}			LLM Prompt Injection	LLM Prompt Injection	LLM Prompt Injection	LLM Prompt Output Components Manipulation		Discover LLM Hallucinations				Exfiltration via AI Inference API	Spanning AI System with Chat Data

<https://atlas.mitre.org/matrices/ATLAS>

- Adversarial Threat Landscape for AI Systems (ATLAS)
- Navigate threats to AI systems through real-world insights
- Covers 15 Tactics with more than 100 techniques.
- Some tactics and techniques adapted from the **MITRE Adversarial Tactics, Techniques, and Common Knowledge (ATT&CK)** matrix
 - a knowledge base that details the different tactics and techniques that adversaries, or cyber attackers, use as well as the platforms they target, based on observations of real attacks that have occurred.

Security Attack on GenAI-powered systems



<https://genai.owasp.org/llm-top-10/>

LLM01: 2025 Prompt Injection LLM01:2025 Prompt Injection A Prompt Injection Vulnerability occurs when user prompts alter the... Read More	LLM02: 2025 Sensitive Information Disclosure LLM02:2025 Sensitive Information Disclosure Sensitive information can affect both the LLM and its application... Read More	LLM03: 2025 Supply Chain LLM03:2025 Supply Chain LLM supply chains are susceptible to various vulnerabilities, which can... Read More	LLM04: 2025 Data and Model Poisoning LLM04:2025 Data and Model Poisoning Data poisoning occurs when pre-training, fine-tuning, or embedding data is... Read More	LLM05: 2025 Improper Output Handling LLM05:2025 Improper Output Handling Improper Output Handling refers specifically to insufficient validation, sanitization, and... Read More
LLM06: 2025 Excessive Agency LLM06:2025 Excessive Agency An LLM-based system is often granted a degree of agency... Read More	LLM07: 2025 System Prompt Leakage LLM07:2025 System Prompt Leakage The system prompt leakage vulnerability in LLMs refers to the... Read More	LLM08: 2025 Vector and Embedding Weaknesses LLM08:2025 Vector and Embedding Weaknesses Vectors and embeddings vulnerabilities present significant security risks in systems... Read More	LLM09: 2025 Misinformation LLM09:2025 Misinformation Misinformation from LLMs poses a core vulnerability for applications relying... Read More	LLM10: 2025 Unbounded Consumption LLM10:2025 Unbounded Consumption Unbounded Consumption refers to the process where a Large Language... Read More

LLM01:2025 Prompt Injection

- Prompt Injection Vulnerability occurs when user prompts alter the LLM's behavior or output in unintended ways.
- Prompt Injection vulnerabilities exist in how models process prompts, and how input may force the model to incorrectly pass prompt data to other parts of the model, potentially causing them to violate guidelines, generate harmful content, enable unauthorized access, or influence critical decisions.
- Developers can build safeguards into system prompts and input handling to help mitigate prompt injection attacks, but effective prevention of jailbreaking requires ongoing updates to the model's training and safety mechanisms.

LLM02:2025 Sensitive Information Disclosure

- Sensitive information can affect both the LLM and its application context.
 - This includes personal identifiable information (PII), financial details, health records, confidential business data, security credentials, and legal documents.
- LLMs, especially when embedded in applications, risk exposing sensitive data, proprietary algorithms, or confidential details through their output.
 - This can result in unauthorized data access, privacy violations, and intellectual property breaches.
- To reduce this risk, LLM applications should perform adequate data sanitization to prevent user data from entering the training model.

LLM03:2025 Supply Chain

- LLM supply chains are susceptible to various vulnerabilities, which can affect the integrity of training data, models, and deployment platforms. These risks can result in biased outputs, security breaches, or system failures.
- Creating LLMs is a specialized task that often depends on third-party models.
 - The rise of open-access LLMs and new fine-tuning methods like, especially on platforms like Hugging Face, introduce new supply-chain risks.
- Carefully vet data sources and suppliers, including T&Cs and their privacy policies, only using trusted suppliers.
 - Regularly review and audit supplier Security and Access, ensuring no changes in their security posture or T&Cs.

LLM04:2025 Data and Model Poisoning

- Data poisoning occurs when pre-training, fine-tuning, or embedding data is manipulated to introduce vulnerabilities, backdoors, or biases.
 - This manipulation can compromise model security, performance, or ethical behavior, leading to harmful outputs or impaired capabilities.
- Data poisoning can target different stages of the LLM lifecycle, including pre-training (learning from general data), fine-tuning (adapting models to specific tasks), and embedding (converting text into numerical vectors).
- Models distributed through shared repositories or open-source platforms can carry risks beyond data poisoning, such as malware embedded through techniques like malicious pickling, which can execute harmful code when the model is loaded
- Track data / model origins and transformations using tools like OWASP CycloneDX or ML-BOM. Verify data legitimacy during all model development stages.

LLM05:2025 Improper Output Handling

- Improper Output Handling refers specifically to insufficient validation, sanitization, and handling of the outputs generated by large language models before they are passed downstream to other components and systems.
- Successful exploitation of an Improper Output Handling vulnerability can result in XSS and CSRF in web browsers as well as SSRF, privilege escalation, or remote code execution on backend systems.
- Treat the model as any other user, adopting a zero-trust approach, and apply proper input validation on responses coming from the model to backend functions.

LLM06:2025 Excessive Agency

- An LLM-based system is often granted a degree of agency by its developer
 - the ability to call functions or interface with other systems via extensions (sometimes referred to as tools, skills or plugins by different vendors) to undertake actions in response to a prompt.
- Excessive Agency is the vulnerability that enables damaging actions to be performed in response to unexpected, ambiguous or manipulated outputs from an LLM, regardless of what is causing the LLM to malfunction.
- Limit the extensions that LLM agents are allowed to call to only the minimum necessary.
- Limit the functions that are implemented in LLM extensions to the minimum necessary.

LLM07:2025 System Prompt Leakage

- The system prompt leakage vulnerability in LLMs refers to the risk that the system prompts or instructions used to steer the behavior of the model can also contain sensitive information that was not intended to be discovered.
- disclosure of the system prompt itself does not present the real risk — the security risk lies with the underlying elements, whether that be sensitive information disclosure, system guardrails bypass, improper separation of privileges, etc.
- Avoid embedding any sensitive information directly in the system prompts. Instead, externalize such information to the systems that the model does not directly access using {placeholder}

LLM08:2025 Vector and Embedding Weaknesses

- Retrieval Augmented Generation (RAG) is a model adaptation technique that enhances the performance and contextual relevance of responses from LLM Applications, by combining pre-trained language models with external knowledge sources.
- Weaknesses in how vectors and embeddings are generated, stored, or retrieved can be exploited by malicious actions (intentional or unintentional) to inject harmful content, manipulate model outputs, or access sensitive information.
- Prevention and mitigation strategies includes: permission and access control, data validation & source authentication, data review for combination & classification, and monitoring and Logging

LLM09:2025 Misinformation

- Misinformation from LLMs poses a core vulnerability for applications relying on these models.
 - Misinformation occurs when LLMs produce false or misleading information that appears credible. This vulnerability can lead to security breaches, reputational damage, and legal liability.
- One of the major causes of misinformation is hallucination—when the LLM generates content that seems accurate but is fabricated.
 - Hallucinations occur when LLMs fill gaps in their training data using statistical patterns, without truly understanding the content.
 - biases introduced by the training data and incomplete information can also contribute to hallucination.
- There are various techniques for prevention and mitigations, depending on the cases.

LLM10:2025 Unbounded Consumption

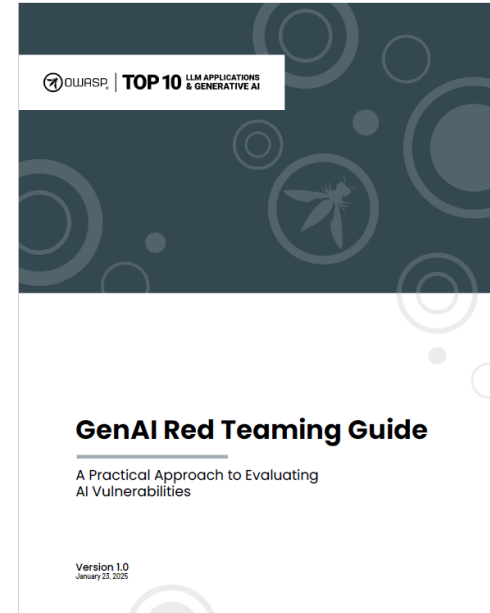
- Unbounded Consumption refers to the process where a Large Language Model (LLM) generates outputs based on input queries or prompts without limit or control.
- Unbounded Consumption occurs when a Large Language Model (LLM) application allows users to conduct excessive and uncontrolled inferences, leading to risks such as denial of service (DoS), economic losses, model theft, and service degradation.
- There are various techniques for prevention and mitigations, depending on the cases.

After implementation and development...

HOW DO WE TEST THE AI SYSTEM?

GenAI Red Teaming Guide

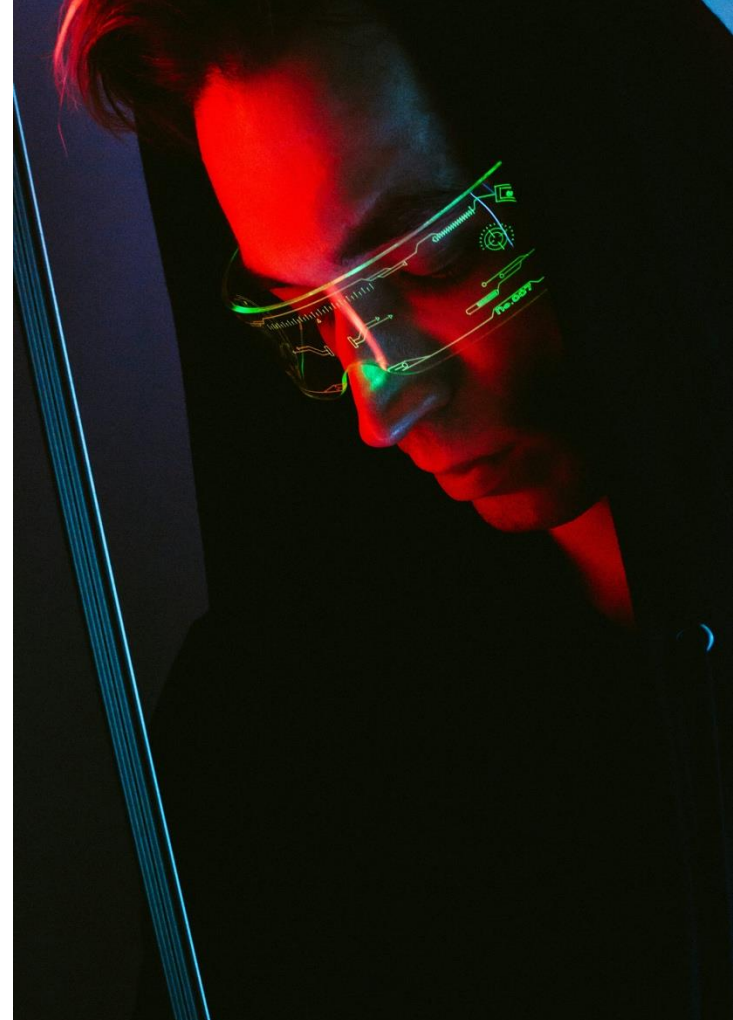
- Published by OWASP. (Jan 2025)
- Outlines the critical components of GenAI Red Teaming, with actionable insights for cybersecurity professionals, AI/ML engineers, software developers, and etc.
- Emphasizes a holistic approach to Red Teaming in four areas:
 - model evaluation,
 - implementation testing,
 - infrastructure assessment (deployment), and
 - runtime behavior analysis (monitoring).



[GenAI Red Teaming Guide - OWASP Gen AI Security Project](#)

GenAI Red Teaming Guide

- to identify vulnerabilities and mitigate risks across AI systems focusing on safety, security, and trust.
- This practice combines traditional adversarial testing with AI specific methodologies, addressing risks like prompt injection, toxic outputs, model extraction, bias, knowledge risks and hallucinations.
- GenAI Red Teaming ensures systems will remain secure, ethical, and aligned with organizational goals.



GenAI Red Teaming Guide

- The guide covers and help to address/identify the following potential risks:
 - **Adversarial Attacks:** Protecting the systems from attacks like prompt injection.
 - **Alignment Risks:** Ensuring AI outputs align with organizational values.
 - **Data Risks:** Protecting against leakage of sensitive or training data.
 - **Interaction Risks:** Preventing unintended harmful outputs or misuse.
 - **Knowledge Risks:** Mitigating misinformation and disinformation.
- organizations must integrate technical methodologies with cross-functional collaboration. Involves multidisciplinary teams, robust engagement frameworks, and iterative processes to adapt to evolving threats and improve system resilience.
- Threat modeling, scenario-based testing, and automated tooling are central to the process, supported by human expertise to address nuanced issues.
- requires continuous monitoring and observability to detect emerging risks, such as model drift or injection attempts.

Summary

- Insecure technology and vulnerabilities in systems will invite malicious cyber intrusions.
- Convenience from AI integration opening up various opportunity for cyber criminals to commit cyber crimes, which leads to major issue.
- Many software manufacturer adopt “release” and “fix” later model.
- These approach resulting wide exposure and open up more vulnerabilities and causes huge damages/lost at the later time.
- Vulnerabilities comes in many forms and variants due to the evolution and emerging of new technologies e.g. GenAI.
- NIST 800-218 (SSDF) - A core set of high-level secure software development practices that can be integrated into each SDLC implementation
- NIST 800-218A (SSDFGenAI) - Augments the secure software development practices and tasks defined in SSDF version 1.1
- ALTAS MITRE is a common matrix to systematically document, classify, and catalog these vulnerabilities
- OWASP Top 10 LLM outline the most common risk, threats, and vulnerability on GenAI (LLM) powered application.
- OWASP GenAI Red Teaming Guide outline holistic approach fro Red Teaming strategies.

References

- NCSC Guidelines for Secure AI System Development
- NIST 800-218 Secure Software Development Framework (SSDF) V 1.1
<https://csrc.nist.gov/projects/ssdf>
- NIST 800-218A Secure Software Development Practices for Generative AI and Dual-Use Foundation Models <https://csrc.nist.gov/pubs/sp/800/218/a/final>
- MITRE Adversarial Threat Landscape for AI Systems (ATLAS) <https://atlas.mitre.org/matrices/ATLAS>
- OWASP Top 10 for LLM Applications
<https://genai.owasp.org/llm-top-10/>
- GenAI Red Teaming Guide
<https://genai.owasp.org/resource/genai-red-teaming-guide/>

